

WEEK 4 SECURITY FINDINGS

Advanced Monitoring, Rate Limiting & Header Security
NodeGoat Application

Student Name	Fatima Haq
Course	Cybersecurity
Project Title	Securing the NodeGoat Application
Report Week	Week 4 — Security Findings Documentation
Date	July 14, 2026
Total Findings	6 (2 High, 4 Medium)
All Fixed	Yes — 6 / 6

CONFIDENTIAL – For Educational / Internship Use Only

1. Security Findings

Finding 1: No Monitoring of Failed Login Attempts

 MEDIUM

Description

The application originally had no mechanism to monitor repeated failed login attempts. Authentication failures occurred silently with no trace left for security review.

Risk

Attackers could repeatedly attempt to guess user credentials without any detection, monitoring, or alerting mechanism in place. This makes brute-force and credential-stuffing attacks invisible to the development and security team.

Severity

Medium

Evidence

No security logs were generated for failed login events before implementation. The application provided no visibility into authentication activity.

Remediation

Winston Logger was implemented to record the following events on every failed login attempt:

- Invalid username attempted
- Invalid password attempted
- Client IP address of the request
- Timestamp of each attempt

```
const logger = require("../logger");

// On invalid username:
logger.warn(`Invalid login attempt - username: ${username} | IP: ${req.ip}`);

// On invalid password:
logger.warn(`Invalid password attempt - username: ${username} | IP: ${req.ip}`);
```

Note: Logging alone does not block attacks — it improves visibility. Rate limiting (Finding 2) is the control that actually limits attack speed.

Status

✓ Fixed

Finding 2: No Rate Limiting

 HIGH

Description

The application allowed unlimited authentication requests with no throttling or blocking mechanism. An attacker could automate thousands of login attempts per minute.

Risk

Without rate limiting, attackers could perform brute-force password attacks — systematically trying every possible password until the correct one is found — without any slowdown or lockout.

Severity

High

Evidence

Unlimited login attempts were possible before implementing rate limiting. No HTTP 429 (Too Many Requests) responses were returned regardless of attempt frequency.

Remediation

The express-rate-limit middleware was installed and configured on the login route:

```
npm install express-rate-limit
const rateLimit = require("express-rate-limit");

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15-minute window
  max: 5, // max 5 requests per window
  message: "Too many login attempts. Please try again after 15 minutes."
});

app.post("/login", loginLimiter, authController.login);
```

Parameter	Value
Maximum Requests	5
Time Window	15 Minutes
HTTP Response	429 Too Many Requests
Reset After	Window expiry (15 min)

Status

✓ Fixed

Finding 3: Missing CORS Configuration

⚠ MEDIUM

Description

Cross-Origin Resource Sharing (CORS) was not configured on the application. Without CORS restrictions, any website could send requests to the application's API endpoints from a browser.

Risk

Unauthorized websites could potentially send cross-origin requests to the application, enabling Cross-Site Request Forgery (CSRF) style attacks where a malicious site tricks a logged-in user's browser into making requests on their behalf.

Severity

Medium

Evidence

The application accepted requests from all origins without any restriction. No Access-Control-Allow-Origin header was configured to limit which domains could interact with the API.

Remediation

The cors middleware was installed and configured to allow requests only from trusted origins:

```
npm install cors

const cors = require("cors");

const corsOptions = {
  origin: ["http://localhost:4000"], // trusted origins only
  methods: ["GET", "POST"],
  credentials: true
};

app.use(cors(corsOptions));
```

Note: In production, replace localhost:4000 with the actual domain of the application. Never use origin: '*' in production environments as it defeats the purpose of CORS.

Status

✓ Fixed

Finding 4: Missing API Authentication

⚠ HIGH

Description

Application endpoints lacked API-level authentication. Any client, whether authorized or not, could send requests to protected resources without proving its identity.

Risk

Unauthorized clients could attempt to access protected resources, scrape application data, or abuse backend functionality without any credential check at the API layer.

Severity

High

Evidence

Requests were accepted without API key verification. Endpoints returned data regardless of whether the requesting client had any form of API-level credentials.

Remediation

A custom API key authentication middleware was implemented. All protected routes now require a valid API key in the request header:

```
// middleware/apiAuth.js
const apiKeyAuth = (req, res, next) => {
  const apiKey = req.headers["x-api-key"];
  if (!apiKey || apiKey !== process.env.API_KEY) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  next();
};

module.exports = apiKeyAuth;
```

Unauthorized requests now receive:

```
HTTP 401 Unauthorized
```

Note: The API_KEY value must be stored in an environment variable (process.env.API_KEY) and must never be hardcoded in source code or committed to version control.

Status

✓ Fixed

Finding 5: Missing Content Security Policy (CSP)

 MEDIUM

Description

The application did not implement a Content Security Policy. Without a CSP, the browser has no guidance on which sources of scripts, styles, and other resources are trusted.

Risk

The browser was more vulnerable to malicious script execution and Cross-Site Scripting attacks. An XSS payload could load external scripts or exfiltrate data to attacker-controlled servers without any browser-level restriction.

Severity

Medium

Remediation

A Content Security Policy was implemented using the Helmet.js middleware:

```
const helmet = require("helmet");

app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ['self'],
    scriptSrc:  ['self'],           // blocks inline and external scripts
    styleSrc:   ['self'],
    imgSrc:     ['self', "data:"],
    objectSrc:  ['none'],
    frameAncestors: ['none']      // also prevents clickjacking
  }
}));
```

Note: The CSP header restricts where scripts can load from. A strict CSP significantly reduces the impact of any remaining XSS vulnerabilities — even if a payload is injected, it cannot load external resources or execute inline scripts.

Status

✓ Fixed

Finding 6: Missing HSTS Header

 MEDIUM

Description

The application did not enforce secure HTTPS communication using the HTTP Strict-Transport-Security (HSTS) header. Without HSTS, browsers may connect to the application over unencrypted HTTP.

Risk

Users could potentially access the application over insecure HTTP connections, exposing session tokens, credentials, and sensitive data to network-level interception (man-in-the-middle attacks).

Severity

Medium

Remediation

HTTP Strict-Transport-Security (HSTS) was enabled using Helmet:

```
app.use(helmet.hsts({
  maxAge: 31536000, // 1 year in seconds
  includeSubDomains: true, // applies to all subdomains
  preload: true
}));
```

Once HSTS is set, browsers will automatically upgrade all future HTTP connections to HTTPS for the configured maxAge duration, even without being redirected.

Note: HSTS is only effective when the application is deployed over HTTPS. In the current local development environment (HTTP), this header is applied in code but will only take effect in a production HTTPS deployment.

Status

✓ Fixed

2. Summary of Findings

Finding	Severity	Status
No Monitoring of Failed Login Attempts	Medium	Fixed
No Rate Limiting	High	Fixed
Missing CORS Configuration	Medium	Fixed
Missing API Authentication	High	Fixed
Missing Content Security Policy (CSP)	Medium	Fixed
Missing HSTS Header	Medium	Fixed

3. Overall Result

All six identified security weaknesses were successfully mitigated. The following controls were implemented during Week 4:

- Winston Logger — records failed login attempts with IP address and timestamp
- express-rate-limit — limits login attempts to 5 per 15-minute window
- CORS middleware — restricts cross-origin requests to trusted origins only
- API Key middleware — blocks unauthorized clients from accessing protected endpoints
- Content Security Policy via Helmet — restricts browser resource loading to trusted sources
- HSTS via Helmet — instructs browsers to enforce HTTPS-only connections

These enhancements significantly improved the overall security of the NodeGoat application and reduced the risk of brute-force attacks, cross-origin abuse, unauthorized API access, XSS escalation, and network-level interception. Combined with the controls from Weeks 2 and 3, the application now implements a layered security posture covering authentication, input handling, transport, headers, monitoring, and access control.

This report was produced as part of a cybersecurity internship training exercise on a deliberately vulnerable application in a local environment. No real or production systems were tested.